Serial Python Minimalis: Tutorial #04-Kontrol Alur

Ahmad R. T. Nugraha

ver. 21 April 2026

Sejauh ini, kode yang telah kita tulis dieksekusi dalam urutan ("alur") yang telah ditentukan sebelumnya, yakni satu pernyataan dieksekusi setelah pernyataan lainnya, atau diulang dalam jumlah yang tetap dalam kasus perulangan `for`. Namun, untuk program-program yang lebih kompleks, sering kali kita memerlukan kode tersebut dapat bercabang, yakni mengeksekusi pernyataan kode yang berbeda berdasarkan satu atau lebih kondisi. Oleh karena itu, kita harus mengetahui beberapa teknik kontrol alur program dalam Python.

1 Perbandingan dan Logika

Dalam Python, kontrol alur di antaranya dapat dicapai menggunakan sintaksis `if...elif...else` dengan perbandingan antarobjek dievaluasi sebagai `True` (Benar) atau `False` (Salah). Daftar operator perbandingan yang berguna disajikan pada Tabel 1. Perhatikan bahwa untuk membandingkan kesetaraan dua objek (misalnya angka), operatornya adalah ==: dua tanda sama dengan. Operator ini jangan sampai disamakan dengan satu tanda sama dengan (texttt=) yang spesifik digunakan untuk penetapan atau penugasan (*assignment*).

Tabel 1: Operator perbandingan Python

Operator	Arti
`==`	Sama dengan
`!=`	Tidak sama dengan
`>`	Lebih besar dari
`<`	Lebih kecil dari
`>=`	Lebih besar dari atau sama dengan
`<=`	Lebih kecil dari atau sama dengan

Mari kita lihat beberapa contoh:

```
a = 4
a == 2 + 2
```

Output:

```
True
```

```
a >= 5
```

Output:

```
False
```

Perbandingan dapat digabungkan dengan kata kunci operator logika `and`, `not`, dan `or`:

```
a, b, c = 1, 2, 3
a > b or c > b
```

Output:

```
True
```

```
a > b and c > b
```

Output:

```
False
```

```
not a > b
```

Output:

```
True
```

Dalam kontrol alur menggunakan operator perbandingan dan logika, ada prioritas urutan (*precedence*) yang perlu diperhatikan. Aturan dasarnya, operasi aritmetika (+, *, dll.) selalu memiliki presedensi lebih tinggi (dievaluasi lebih dulu) daripada perbandingan (==, <, dll.), dan perbandingan selalu memiliki presedensi lebih tinggi daripada operator logika. Di antara operator logika, presedensinya dari `not` (tertinggi) hingga `or` (terendah). Seperti halnya aritmetika, ekspresi di dalam tanda kurung memiliki presedensi tertinggi dari semuanya.

```
not a == 1 and b > c
```

Output:

```
False
```

```
not (a == 1 and b > c)
# (not True) and False => False
```

```
# not (True and False) => (not False) => True
```

Output:

```
True
```

Ada beberapa "operator keanggotaan" (*membership operators*) yang kerap berguna untuk keseharian pemrograman kita, di antaranya `in` dan `not in`, masing-masing menguji keanggotaan dan ketiadaan anggota dalam sebuah barisan:

```
metals = 'Sc Ti V Cr Mn Fe Co Ni Cu Zn'.split()
metals
```

Output:

```
['Sc', 'Ti', 'V', 'Cr', 'Mn', 'Fe', 'Co', 'Ni', 'Cu', 'Zn']
```

```
'Fe' in metals
```

Output:

```
True
```

```
'V' not in metals
```

Output:

```
False
```

Dua fungsi bawaan lainnya berguna untuk perbandingan yang melibatkan semua item dalam sebuah barisan: `any` mengembalikan `True` jika salah satu item setara dengan `True`; `all` mengembalikan `True` jika semuanya setara dengan `True`:

```
all(['a', True, 2])
```

Output:

```
True
```

```
any([0, [], '', False])
```

Output:

```
False
```

2 Konstruksi `if ... elif ... else`

Kode di dalam blok kode `if` (diindentasi 4 spasi, seperti perulangan `for`) hanya dieksekusi jika ekspresi setelah kata kunci `if` terevaluasi sebagai `True`:

```
a = 3.14159
if a > 3:
    print('a is greater than 3')
```

Output:

```
a is greater than 3
```

```
if a > 4:
    print('a is greater than 4')
```

Kata kunci `else` menyediakan jalur alternatif jika ekspresinya tidak `True`:

```
if a < 3:
    print('a is less than 3')
else:
    print('a is greater than 3')
```

Output:

```
a is greater than 3
```

Kata kunci `elif` kemudian memungkinkan beberapa kondisi untuk diperiksa. Blok kode yang mengikuti perbandingan `True` pertama akan dieksekusi:

```
if a < 3:
    print('a is less than 3')
elif a < 4:
    print('a is between 3 and 4')
else:
    print('a is greater than 4')
```

Output:

```
a is between 3 and 4
```

`True` dan `False` adalah nilai boolean konstan bawaan Python (bertipe `bool`). Nilai-nilai ini tidak dapat diubah atau ditetapkan sebagai nama variabel. Untuk tujuan perbandingan, Python juga memiliki konsep *truthiness* (kebenaran), yakni bahwa objek non-boolean diperlakukan sebagai `True` atau `False` berdasarkan nilainya. Secara khusus, `float` atau `int` dengan nilai 0 setara dengan `False` (semua nilai lainnya adalah `True`); string kosong (’’) dan list kosong (`[]`) bernilai `False` (string atau list apa pun yang memiliki isi setara dengan `True`). Konsep inimemungkinkan sintaksis seperti:

```
b = 10
if b % 2:
    print('b is odd')
else:
    print('b is even')
```

Output:

```
b is even
```

Di sini, jika `b` ganjil, `b % 2` adalah 1 (yaitu, `b` mod 2 = 1, sisa bagi dari `b/2`) yang setara dengan `True`. Jika `b` genap, maka `b % 2` adalah 0, yang setara dengan `False`.

3 Perulangan `while`

Cara alternatif untuk membuat perulangan adalah dengan kata kunci `while`. Dari pelajaran sebelumnya, kita tahu bahwa perulangan `for` dieksekusi untuk jumlah iterasi yang tetap. Di sini kita akan lihat bahwa perulangan `while` hanya akan mengeksekusi blok kode selama kondisinya masih terpenuhi (`True`). Misalnya:

```
i = 0
while i < 3:
    i += 1
    print(i)
```

Output:

```
1
2
3
```

Pencacah di sini (*counter*) `i` diinisialisasi ke 0, yang tentunya lebih kecil dari 3, sehingga bagian badan perulangan `while` dieksekusi. Pertama-tama `i` bertambah dan nilainya (1) dicetak ke layar. Eksekusi kemudian kembali ke pengujian perulangan `while`. Semua ini berulang sampai `i` mencapai nilai 3, yang pada situasi ini nilainya dicetak, tetapi sekarang pengujian perulangan `while`, `i < 3`, memberikan hasil `False` sehingga eksekusi kode keluar atau selesai dari bagian badan perulangan tersebut.

4 Kontrol Alur Lainnya: `break`, `continue`, dan `pass`

Ketiga kata kunci ini memberikan kontrol lebih lanjut terhadap eksekusi perulangan. Saat pernyataan `break` tercapai, perulangan akan segera dihentikan:

```
for i in range(10):
    print(i)
    if i == 3:
```

```
        break
```

Output:

```
0
1
2
3
```

Objek `range(10)` akan menghasilkan angka 0, 1, 2, ..., 9 dan mengumpankannya ke `i` secara bergantian, tetapi ketika `i` mencapai 3, maka perulangan diakhiri. Penggunaan `break` yang umum adalah untuk keluar dari perulangan tak terbatas (*infinite loop*):

```
n = 0
while True:
    n += 1
    if not (n % 12 or n % 42):
        break
print(n)
```

Output:

```
84
```

Kode di atas mencetak angka terkecil, yakni kelipatan persekutuan terkecil (KPK), yang habis dibagi oleh 12 dan 42.

Ketika pernyataan `continue` ditemui, iterasi perulangan saat ini akan ditinggalkan dan iterasi berikutnya (jika ada) dimulai. Seperti `break`, kata kunci `continue` dapat digunakan di dalam perulangan `for` maupun `while`:

```
for n in range(10):
    if n % 3:
        continue
    print(n)
```

Output:

```
0
3
6
9
```

Di sini, pernyataan `print` hanya dieksekusi jika `n % 3` bernilai 0 (setara dengan `False`), yaitu jika 3 membagi `n`. Jika tidak, `n % 3` setara dengan `True` dan pernyataan `continue` dieksekusi, segera mengembalikan alur kode ke bagian atas perulangan `for`, yang kemudian menambah nilai `n`.

5 Eksepsi/Pengecualian (*Exceptions*)

Kondisi kesalahan yang ditemui dalam kode Python dilaporkan sebagai pengecualian (*exceptions*), yang biasanya disertai laporan pelacakan balik (*traceback*) yang dapat membantu melacak penyebab kesalahan. Misalnya:

```
print(float("2.3D-2"))
```

Output:

```
ValueError                                Traceback (most recent call last)
Input In [x], in <module>
----> 1 print(float("2.3D-2"))

ValueError: could not convert string to float: '2.3D-2'
```

Exception yang dilaporkan di sini adalah `ValueError`: objek yang diteruskan ke fungsi `float` adalah sebuah string, `"2.3D-2"`, yang merupakan tipe yang benar, tetapi nilainya mengandung karakter yang tidak valid (`'D'`), sehingga `float` tidak dapat mengubahnya menjadi bilangan riil titik kambang (*floating-point*). Beberapa *Exception* yang umum ditemui dapat dibaca pada Tabel 2.

Tabel 2: Pengecualian (Exceptions) Python yang umum

Exception	Penyebab dan deskripsi
`IndexError`	Mengindeks sebuah urutan (seperti list atau string) dengan subskrip yang berada di luar rentang.
`KeyError`	Mengindeks sebuah kamus (*dictionary*) dengan kunci yang tidak ada di dalam tersebut. Struktur kamus akan kita pelajari belakangan.
`NameError`	Merujuk ke nama variabel yang belum didefinisikan.
`TypeError`	Mencoba menggunakan objek dengan tipe yang tidak sesuai sebagai argumen untuk operasi atau fungsi bawaan.
`ValueError`	Mencoba menggunakan objek dengan tipe yang benar tetapi dengan nilai yang tidak kompatibel sebagai argumen untuk operasi atau fungsi bawaan.
`ZeroDivisionError`	Mencoba membagi dengan nol (baik secara eksplisit menggunakan "/" atau "//", maupun sebagai bagian dari operasi modulo "%").
`SyntaxError`	Menggunakan sintaksis Python yang keliru, misalnya mencoba menetapkan kata kunci tercadang (*reserved*) sebagai nama variabel, menghilangkan tanda kurung tutup, atau menggunakan satu tanda sama dengan (=) untuk perbandingan alih-alih ==.

Python yang idomatik menganut filosofi yang diringkas dengan singkatan EAFP: *"Easier to Ask Forgiveness than to seek Permission"* ("Lebih Mudah Meminta Maaf daripada Meminta Izin"). Daripada mencoba menghindari *Exceptions*, pada praktiknya sangat umum untuk tetap menjalankan kode tersebut, tetapi di dalam sebuah blok `try`, yang diikuti oleh blok `except` yang menangani setiap kesalahan yang diantisipasi. Misalnya:

```
x = 0
try:
    print(4/x)
except ZeroDivisionError:
    print("4/0 is not defined!")
```

Output:

```
4/0 is not defined!
```

Eksekusi kode kemudian berlanjut secara normal setelah blok `try...except` ini, terlepas dari apakah nilai $4/x$ ataupun pesan peringatannya yang dicetak. Kesalahan yang tidak terantisipasi diharapkan untuk menghentikan eksekusi kode dan dilaporkan kepada pemrogram/pengguna.

6 Studi Kasus

Kasus 1 Wujud Zat Logam

Diketahui data berikut mengenai titik leleh (*melting points*) dan titik didih (*boiling points*) standar dari berbagai logam.

```
metals = ['Fe', 'Al', 'Hg', 'Ti', 'Pb']
# Suhu perubahan wujud standar untuk setiap logam, dalam Kelvin.
melting_points = [1811, 933, 234, 1941, 601]
boiling_points = [3134, 2743, 630, 3560, 2022]
```

Tentukan wujudnya (padat, cair, atau gas) pada suhu 500°C.

Solusi

Pertama, konversikan suhu rujukan, `T`, ke Kelvin, lalu iterasi ketiga `list` tersebut secara bersamaan. Untuk setiap logam, bandingkan `T` dengan titik didih (`Tb`) dan titik lelehnya (`Tm`).

```python
# Suhu (K)
T = 500 + 273

for metal, Tm, Tb in zip(metals, melting_points, boiling_points):
    if T > Tb:
        print(f'{metal} is a gas at {T} K')
    elif T > Tm:
        print(f'{metal} is a liquid at {T} K')
    else:
        print(f'{metal} is a solid at {T} K')
```

Output:

```
Fe is a solid at 773 K
Al is a solid at 773 K
Hg is a gas at 773 K
Ti is a solid at 773 K
Pb is a liquid at 773 K
```

Untuk menghasilkan `list` yang berisi nilai boolean, `True` atau `False`, yang menunjukkan apakah `T` lebih kecil dari titik didih, kita dapat menggunakan metode append:

```python
is_gas = []
for Tb in boiling_points:
    if T > Tb:
        is_gas.append(True)
    else:
        is_gas.append(False)
is_gas
```

Output:

```
[False, False, True, False, False]
```

Namun, kita dapat sepenuhnya menghilangkan blok `if...else` tersebut. Karena kita menambahkan hasil dari perbandingan `T > Tb` (`True` atau `False`), cara yang lebih baik adalah:

```python
is_gas = []
for Tb in boiling_points:
    is_gas.append(T > Tb)
is_gas
```

Output:

```
[False, False, True, False, False]
```

Bahkan lebih baik lagi, kita dapat membuat `list` ini tanpa harus menggunakan metode `append`: gunakan *list comprehension* untuk menghasilkan list tersebut:

```
is_gas = [T > Tb for Tb in boiling_points]
is_gas
```

Output:

```
[False, False, True, False, False]
```

Fungsi bawaan `any` dan `all` masing-masing dapat digunakan untuk memastikan apakah ada salah satu atau seluruh logam yang berwujud gas pada suhu `T`:

```
any(is_gas), all(is_gas)
```

Output:

```
(True, False)
```

Kasus 2 Volume Molar Gas van der Waals

Tentukan volume molar, V_m, dari uap heksana pada $T = 25°C$ dan $p = 10\,\text{kPa}$. Gunakan persamaan keadaan van der Waals dengan parameter $a = 2.471\ \text{m}^6\ \text{Pa mol}^{-2}$ dan $b = 1.735 \times 10^{-4}\ \text{m}^3\ \text{mol}^{-1}$.
Persamaan keadaan van der Waals adalah

$$p = \frac{RT}{V_m - b} - \frac{a}{V_m^2}$$

Solusi

Pikiran pertama kita mungkin langsung mengatur ulang persamaan ini untuk V_m:

$$pV_m^3 - (bp + RT)V_m^2 + aV_m - ab = 0$$

Sayangnya, ini adalah persamaan kubik dalam V_m, dan tidak ada cara mudah untuk menemukan akar (yang relevan) secara langsung.
Namun, ada pendekatan numerik yang didasarkan pada iterasi. Pengaturan ulang yang berbeda memberikan:

$$V_m = b + \frac{RT}{p + \frac{a}{V_m^2}}$$

dengan suku interaksi antarmolekul, $\frac{a}{V_m^2} \ll p$. Jika kita mulai dari tebakan awal, $V_m^{(0)}$, kita dapat mengharapkan pendekatan yang secara progresif menjadi lebih

baik ke nilai V_m sebenarnya dari proses iteratif:

$$V_m^{(n+1)} = b + \frac{RT}{p + \frac{a}{(V_m^{(n)})^2}}$$

Tebakan awal yang wajar adalah $V_m^{(0)} = \frac{RT}{p}$, yaitu volume molar yang diprediksi oleh persamaan gas ideal.
Definisikan konstanta dan parameter yang kita butuhkan:

```
# Konstanta gas, dalam J.K-1.mol-1.
R = 8.314
# Parameter Van der Waals untuk heksana: m6.Pa.mol-2 dan m3.mol-1.
a, b = 2.471, 1.735e-4
# Suhu (K) dan tekanan (Pa) untuk soal ini.
T, p = 25 + 273.15, 10 * 1000

# Volume molar gas ideal, dalam m3:
Vm = R * T / p
Vm
```

Output:

```
0.0024788191
```

Kita akan mengiterasi sampai nilai V_m berubah kurang dari jumlah yang sangat kecil, `TOL`.

```
TOL = 1.e-12
Vold = R * T / p

while True:
    Vm = b + R * T / (p + a / Vold**2)
    print(Vm)
    if abs(Vm - Vold) < TOL:
        break
    Vold = Vm

print()
print('Vm = {:.4f} L.mol-1'.format(Vm * 1000))
```

Output:

```
0.00023364429019125273
0.00017404750299408598
0.0001738038466444478
0.00017380299660870434
0.00017380299364529655
0.00017380299363496549

Vm = 0.1738 L.mol-1
```

Terakhir, kita harus memeriksa bahwa prosedur ini benar-benar konvergen ke jawaban yang tepat dengan memasukkan nilai V_m ini kembali ke persamaan gas van der Waals:

```
R * T / (Vm - b) - a / Vm**2
```

Output:

```
9999.990276411176
```

Hasil ini sangat dekat dengan nilai yang diharapkan, yaitu $p = 10$ kPa.

Kasus 3 Verifikasi Konfigurasi Elektron

Diberikan sebuah string yang mewakili konfigurasi elektron suatu atom dalam bentuk `'1s2.2s2.2p4'`. Tuliskan sebuah program untuk memverifikasi bahwa konfigurasi tersebut valid. Pengujian berikut harus dilakukan:

1. Orbital harus benar-benar ada (contoh: `'1s2.2d3'` tidak valid).
2. Orbital tidak boleh diduplikasi (contoh: `'1s2.2s1.2s1'` tidak valid).
3. Orbital tidak boleh berisi elektron lebih dari jumlah maksimum yang diizinkan untuk orbital tersebut.

Solusi

Konfigurasi orbital atom ditulis sebagai nl^N dengan $n = 1, 2, 3, \ldots$ adalah bilangan kuantum utama, $l = 0, 1, \ldots, n-1$ adalah bilangan kuantum momentum sudut orbital yang ditulis sebagai huruf (`s`, `p`, `d`, dll.), dan N adalah keterisian orbital. Degenerasi spasial orbital adalah $2l + 1$, dan masing-masing bilangan kuantum proyeksi spin elektron dapat bernilai $m_s = \pm\frac{1}{2}$ sehingga nilai maksimum N adalah $2(2l + 1)$.
Kita akan menyusun kodenya langkah demi langkah, dan mengujinya dengan

string konfigurasi yang (sangat) tidak valid. Karena setiap orbital yang terisi dipisahkan oleh karakter `'.'`, kita bisa mengubahnya menjadi list dengan metode `split`:

```
# Sebuah konfigurasi atom yang sengaja disalahkan.
s = '1p2.2s2.2p6.2s3.4d10'
s.split('.')
```

Output:

```
['1p2', '2s2', '2p6', '2s3', '4d10']
```

Selanjutnya, urai (*parse*) string orbital yang terisi untuk mengekstrak n, l, dan N. Karena huruf `l` terlihat mirip dengan angka `1` di beberapa font, kita akan menamai variabelnya `ell`; untuk lebih menghindari kebingungan, bilangan kuantum utama dinamai `n`, dan jumlah elektron N dinamai `n_electrons`.

```
# Pisahkan string untuk setiap orbital; dan lakukan loop
for occ_orbital in s.split('.'):
    # Orbital adalah dua karakter pertama dari substring
    # Catatan: ini berarti kita tidak bisa memiliki n > 9
    orbital = occ_orbital[:2]
    n = int(orbital[0])
    ell_letter = orbital[1]

    # Cara termudah mengonversi huruf ke integer ell: temukan
    ↪ indeksnya
    ell = 'spdf'.index(ell_letter)
    n_electrons = int(occ_orbital[2:])
    print(f'n={n}, l={ell} ("{ell_letter}"), N={n_electrons}')
```

Output:

```
n=1, l=1 ("p"), N=2
n=2, l=0 ("s"), N=2
n=2, l=1 ("p"), N=6
n=2, l=0 ("s"), N=3
n=4, l=2 ("d"), N=10
```

Hasilnya terlihat bagus. Sekarang periksa apakah $l < n$ dan $N \leq 2(2l+1)$, serta simpan bagian orbital dari konfigurasi (tanpa jumlah keterisiannya) agar kita bisa memastikan bahwa orbital yang sama tidak muncul dua kali:

```python
s = '1p2.2s2.2p6.2s3.4d10'
orbitals = []

for occ_orbital in s.split('.'):
    orbital = occ_orbital[:2]
    n = int(orbital[0])
    ell_letter = orbital[1]
    ell = 'spdf'.index(ell_letter)
    n_electrons = int(occ_orbital[2:])

    # Periksa keberadaan orbital (l harus lebih kecil dari n)
    if ell >= n:
        print(f'No such orbital: {orbital}')

    # Periksa batas maksimum elektron
    if n_electrons > 2 * (2 * ell + 1):
        print(f'Too many electrons in orbital: {occ_orbital}')

    # Periksa duplikasi orbital
    if orbital in orbitals:
        print(f'Repeated orbital: {orbital}')

    orbitals.append(orbital)
```

Output:

```
No such orbital: 1p
Too many electrons in orbital: 2s3
Repeated orbital: 2s
```